gtfs-ingest: The Whole Workflow

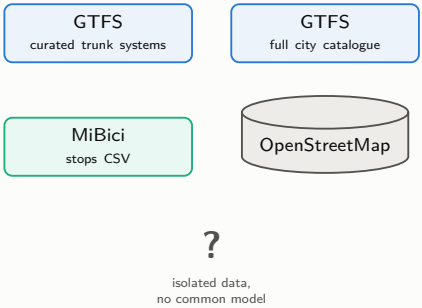
From GTFS feeds, a stops CSV and OpenStreetMap to stations, routable networks and a timetable — Guadalajara Metropolitan Area

Víctor Rodríguez
November 1, 2026

A visual companion to `docs/guide/pipeline-guide.md` · every number and chart from the run `outputs/out_mibici_v12` (2 GTFS feeds + MiBici, OSM network walk, 300 m proximity, typical weekday per feed)

The problem

- Guadalajara's public transport is **several systems**: Mi Macro (BRT), Mi Tren (light rail), SiTren, Mi Transporte buses, and MiBici bike share.
- Their data exists but is **scattered**: different files, different formats, the same stations published twice under different ids.
- There is no **unified, queryable model** of the network: where do lines connect? How often do they run? Which station can I walk to? How long does a trip take?

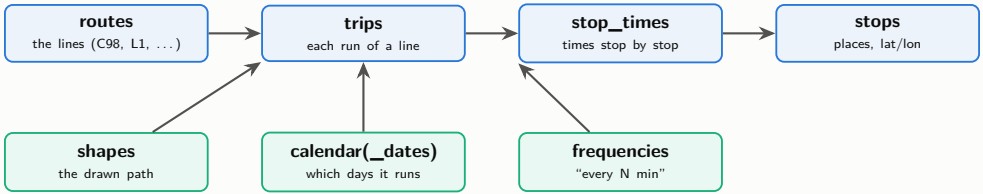


Goal

Build that model automatically from open data, reproducibly, and use it to **compare routing algorithms** — Dijkstra and transit-specific ones (CSA, RAPTOR).

What is a GTFS feed?

The General Transit Feed Specification: a `.zip` of related tables



Source	What it is	Routes	Stops
`improved_gtfs_mimacro_mitren_sitren`	curated trunk systems, calendar 2020–2022	40	1,634
`mdb_2366_202502241922`	the whole city network, calendar 2024	276	12,231
`mibici`	bike-share docks: `id`, `name`, `latitude`, `longitude`	—	484
OpenStreetMap	streets and transit nodes, in PostGIS		

What the pipeline does

Four phases — each command consumes what the previous one wrote

Phase 1

`ingest`

Validate every source and load a clean stop inventory into PostGIS

steps 1–5

Phase 2

`analyze-frequency`

Headways, stations, route graph, routable network, timetable

steps 6–17

Phase 3

`detect-interchanges`
`conflate-osm`

Interchanges, OSM coordinate checks, side of street

steps 18–22

Phase 4

`google-qa`
`locate-stations`

Opt-in external cross-checks, verdicts only

steps 24

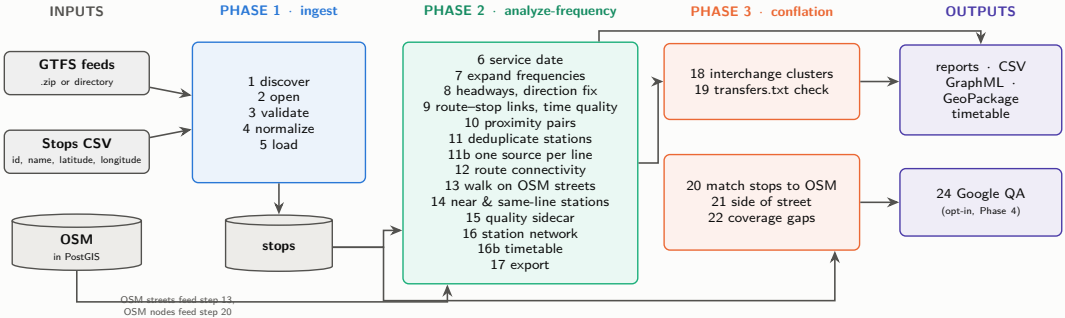
14,349
stop records

10,203
physical stations

249
lines in the routing network

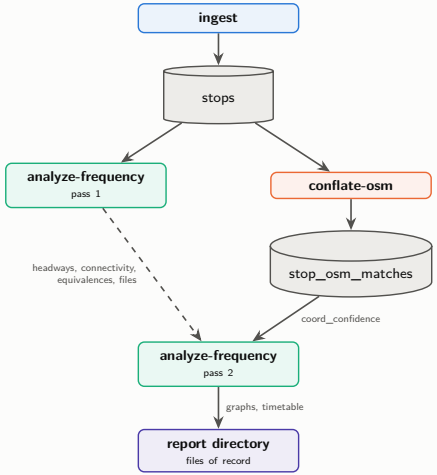
3.29 M
timetable connections

The pipeline at a glance



Run order and data lineage

Why `analyze-frequency` runs twice



- Each command reads what an earlier one wrote.
- The quality sidecar needs OSM verdicts that only `conflate-osm` writes ⇒ a second, cached `analyze-frequency` pass folds them in.
- `detect-interchanges` is independent and can run anywhere after ingest.

```
ingest               --feeds $FEEDS
analyze-frequency    --feeds $FEEDS \
    --network-walk --proximity-m 300
detect-interchanges --feeds $FEEDS
conflate-osm
analyze-frequency    ...    # pass 2
```

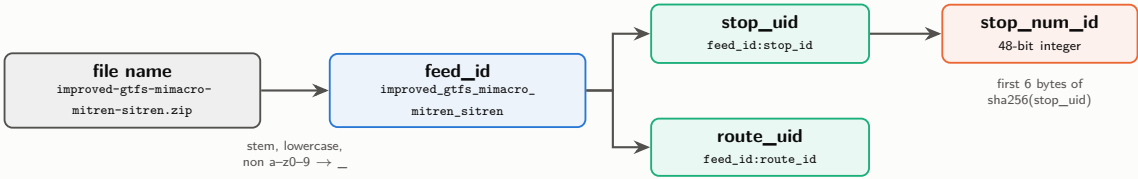
Concepts

Rules every step follows

Identity and provenance · coordinate systems · failure isolation
idempotent re-runs · records versus stations

Identity and provenance

Every identifier is qualified by the source it came from



Guarantees

- `feed_id` never contains `:`, so `stop_uid` splits back cleanly
- A CSV `S1` and a GTFS `S1` coexist — different `stop_uid`
- Same path ⇒ same id, run after run

Collisions

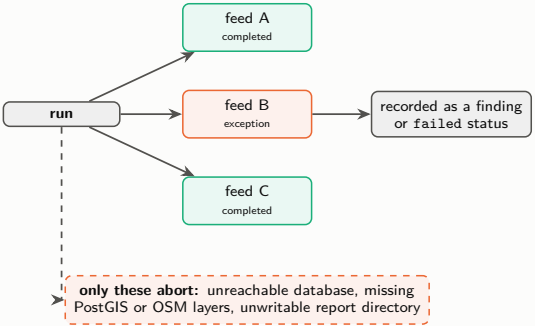
- Two sources with the same name: add 8 hex characters of `sha256(path)`
- `stop_num_id` collision odds $\approx n^2/2^{49}$ — about 10^{-4} at 150,000 stops; a unique index turns one into a load error, never silent corruption

Coordinate systems and failure isolation

Which coordinate system, where

Use	CRS
Data at rest: database and files	EPSG:4326
Planar maths: DBSCAN, local proximity, walk graph, side of street	UTM, auto-detected (32613)
Authoritative distances	PostGIS `geography`
OSM layers	read from the data (3857)

A broken feed never stops a run



Idempotency: how a re-run replaces data

Re-running any command is always safe — but the strategy depends on the table

Feed-scoped

DELETE rows of this feed, then insert — one transaction

`stops`
`route_stop_headways`
`route_headways`

Run-scoped derived

Truncate the whole table, then reload

`route_connections`
`stop_equivalences`
`interchanges`
`stop_osm_matches` · `side of street`

Budgeted verdicts

Upsert row by row — a partial re-check keeps earlier verdicts

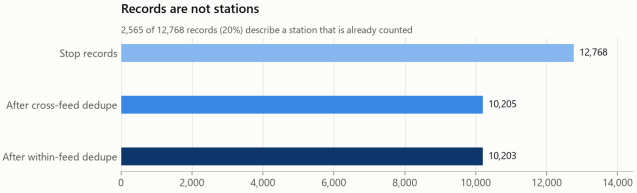
`google_qa_interchange`
`google_qa_stop`

Consequence

For run-scoped tables, pass **every** source you care about in one run: a run with two feeds leaves only those two feeds' connections in `route_connections`.

Records are not stations

Deduplication records equivalences — it never deletes a row



A raw `COUNT(*)` counts **records**. To count **physical stations**, resolve each record to its group with `station_dedup` from `near_stations_quality.csv`:

```
COUNT(DISTINCT COALESCE(
  NULLIF(station_dedup, ''),
  stop_uid))
```

12,768 served records are 10,203 stations: any density or coverage figure on raw records is inflated by $\approx 1.25\times$.

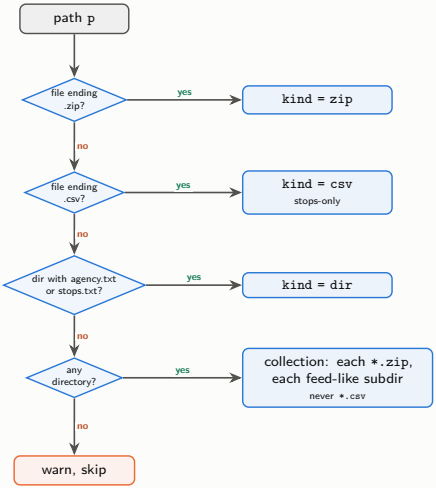
Phase 1 · Ingest

`ingest` — a clean, validated stop inventory in PostGIS

1 Discover sources **2** Open each source **3** Validate **4** Normalize stops **5** Load into PostGIS

STEP 1 Discover sources

Every path is classified once, and the kind is fixed at that moment



Output: a list of `FeedSource(feed_id, path, kind)`, de-duplicated by resolved path.

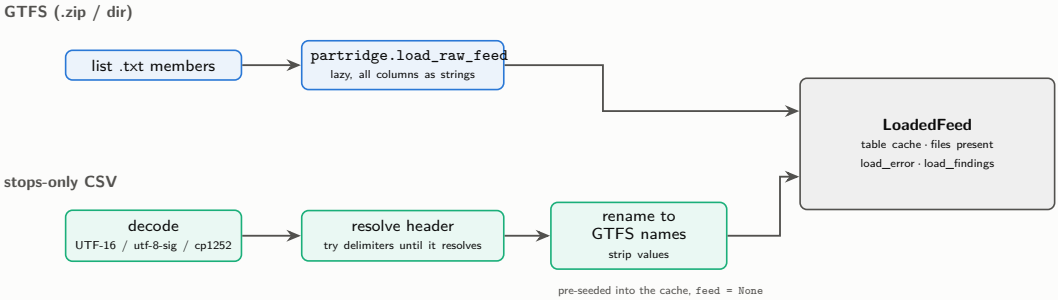
Why a CSV must be named explicitly

The pipeline's own `near_stations.csv` has `ID`, `stop_lat` and `stop_lon` — a *valid* stops CSV. Scanning folders for CSVs would silently re-ingest a previous run's output as a second copy of the network.

A directory named `foo.zip` stays a directory: the kind comes from the layout, not the suffix.

STEP 2 Open each source

Two ways in, one object out — every later step reads both the same way



Why read raw strings?

Validation must see the file *exactly as published*. Typing happens later, explicitly, in normalization.

Why pre-seed the CSV?

Table access consults the cache before the partridge object, so validators, the normalizer and Phase 2 read a CSV source with **no special cases**.

STEP 2 The stops-CSV reader: forgiving, never guessing

Header resolution

```
normalize(cell) =
    strip BOM and quotes
    -> NFKD, drop accents -> casefold
    -> non-alphanumeric runs -> "_"

for each delimiter candidate:
    parse (all values as strings)
    map normalize(cell) -> canonical
    if stop_id, stop_lat, stop_lon found:
        accept          # first match wins
# none resolved -> ERROR unresolved_csv_header
```

Examples: `Latitud`, `LATITUDE`, `lat` → `stop_lat`
`ESTACIÓN` → `stop_name`

Canonical	Accepted spellings
`stop_id`	id, stop_id, stopid, codigo, clave
`stop_name`	name, stop_name, stopname, nombre, estacion
`stop_lat`	latitude, lat, stop_lat, latitud
`stop_lon`	longitude, lon, lng, long, stop_lon, longitud

- **utf-8-sig first:** Spanish-locale Excel writes ;-separated UTF-8 with a BOM
- **Decimal commas** (20,6736) are flagged, never rewritten
- Unknown columns such as MiBici's `dpcapacity` are ignored
- The error names what was found, what is missing, and the spellings that would work

STEP 3 Validate

Every check runs — a crashing check becomes a finding, not a stopped run

	Check	Finding
ERROR	agency, stops, routes, trips, stop_times present	`missing_required_file`
ERROR	calendar.txt or calendar_dates.txt present	`missing_calendar`
ERROR	present files parse; required files non-empty	`unreadable_file`
ERROR	required columns per file	`missing_required_columns`
ERROR	stops: non-null, unique `stop_id`	`duplicate_stop_ids`
ERROR	stops: coordinates in WGS84 range	`invalid_coordinates`
WARN	`parent_station` references a known stop	`dangling_parent_station`
ERROR	trips→routes, stop_times→trips, stop_times→stops	`dangling_reference`
WARN	declared service ended before the analysis date	`feed_expired`
INFO	optional-file inventory	`optional_files`

Severity policy

ERROR spec violation or data loss
WARNING recoverable
INFO inventory

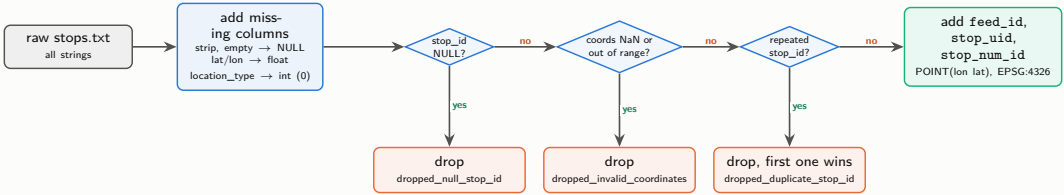
`--fail-on-error` exits 2 on any ERROR.

Stops-only sources

The required-files check becomes one INFO `stops_only_source`. Every other check still runs — a bad coordinate in a CSV raises the same finding as in a GTFS feed.

STEP 4 Normalize stops

Repair by exclusion only — a value is never guessed



Output columns

`stop_uid, stop_num_id, feed_id, stop_id, stop_name, stop_lat, stop_lon, location_type, parent_station, geometry`

Drop counts mirror the validator codes, so

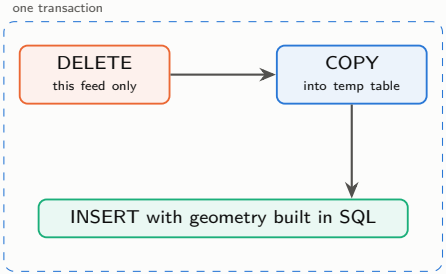
`source rows - dropped = normalized rows`

is always reconstructible from the report.

STEP 5 Load stops into PostGIS

Per-feed replace, bulk-copied, all inside one transaction

```
BEGIN;
DELETE FROM stops WHERE feed_id = :feed_id;
CREATE TEMP TABLE tmp_stops (...)
    ON COMMIT DROP;
COPY tmp_stops FROM STDIN;          -- bulk
INSERT INTO stops (..., geom)
SELECT ..., ST_SetSRID(
         ST_MakePoint(stop_lon, stop_lat), 4326)
FROM tmp_stops;
COMMIT;
```



- `COPY` fails? The same rows go through chunked `executemany` (1,000 per chunk) — same transaction
- Indexes: GiST on `geom`, `feed_id`, unique `stop_num_id`
- Other feeds are never touched

Phase 2 · Service analysis

`analyze-frequency` — per feed, then across all feeds

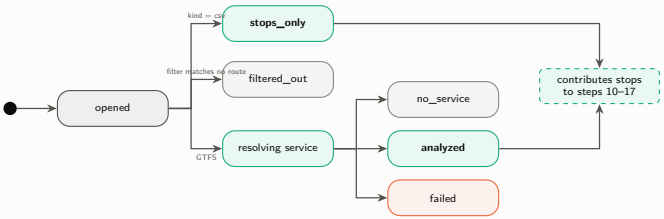
Per feed: 6 service date 7 expand frequencies 8 headways 9 route–stop links & geometry

Across feeds: 10 proximity 11 dedupe 11b one source per line 12 connectivity 13 OSM walk

14 near stations 15 quality 16 station network 16b timetable 17 export

The life of one feed in Phase 2

Each feed runs inside its own failure boundary



Stops-only sources branch out first

No calendar and no trips, so every service-date path would fail. Instead they are normalized, loaded and marked `stops_only` with no service date — and still take part in steps 10–17.

- Exempt from `--transport-mode` / `--route`: nothing to match
- Excluded from `--common-date`: no calendar to share
- No service date, so they never make the run look incoherent

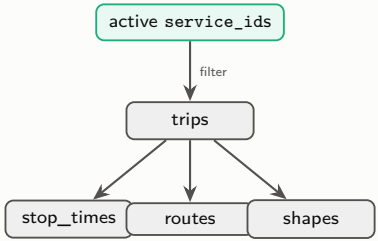
STEP 6 Resolve the service date

Headways depend on which trips actually run that day

Active services on a date

$$active(d) \;=\; \{\, s : \text{weekday flag set},\ start \le d \le end \,\} \;\cup\; \{\, s : \text{added on } d \,\} \;\setminus\; \{\, s : \text{removed on } d \,\}$$

Mode	Date chosen
`--service-date D`	exactly D
default: `--date-rule typical-weekday`	per feed: most common Mon–Fri service on days `calendar_dates` doesn't touch; middle Tue–Thu date
`--date-rule busiest`	the date with the most trips — a **holiday** here
`--feed-service-date F=D`	pin one feed
`--common-date`	$\max\left(\bigcap_{\text{feeds}} \{\text{served dates}\}\right)$; exit 1 if empty
`--service-date-range`	every served date in the window, $\le$ 31 days

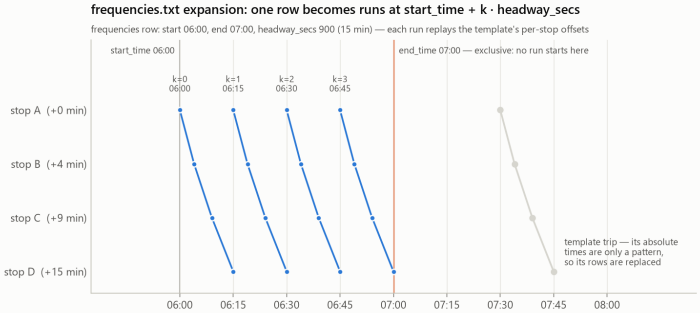


the filter cascades through foreign keys

"Busiest" picked the holidays 2020-01-01 and 2024-01-01 (Sunday service on top of weekday service). Typical weekday: **2021-05-04** (improved), **2024-06-27** (mdb); the calendars never overlap, which `service_dates_note` explains.

STEP 7 Expand frequencies.txt

A frequency-defined trip is a pattern, not a departure



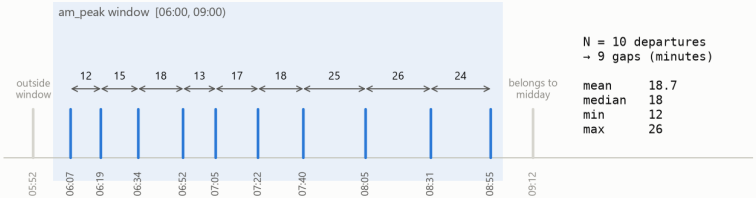

```
n_runs  = min(ceil((end - start) / headway), 1000)
offsets = departure(stop) - first departure
for k in 0 .. n_runs-1:
    emit (trip, stop, start + k*headway + offset)
drop duplicate rows; replace the template rows
```

- `end_time` is exclusive; invalid rows skipped with a warning
- Duplicates dropped: overlapping ranges would create 0-second gaps
- On the analysed day, **82 of 84** (improved) and **500 of 511** (mdb) trips are frequency templates

STEP 8 Compute headways

The gap between consecutive departures, per stop, route, direction and window

Headways at one stop: gaps between consecutive departures inside one window



Definition

Window $w = [start, end)$ · N departures $\Rightarrow N - 1$ gaps
`mean, median, min, max` of the gaps, in seconds
$N = 1$: headway undefined, statistics NULL

Details

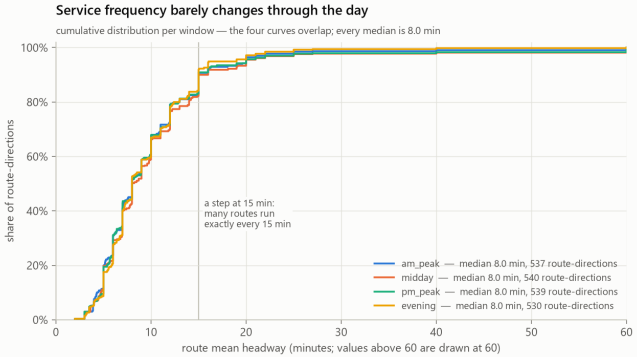
Default windows: am_peak 06–09, midday 09–15, pm_peak 15–19, evening 19–23
Blank `direction_id` → −1; directions never merged
Mixed-sense routes relabelled first (next slide)

STEP 8 Route rollup — and what the data shows

Per route, direction and window

`mean_headway_s`	mean of stop-level means
`stop_count`	stops with a departure
`stops_with_headway`	stops with ≥ 2 departures
`trip_count`	max stop-level count

Unweighted: every stop counts equally.

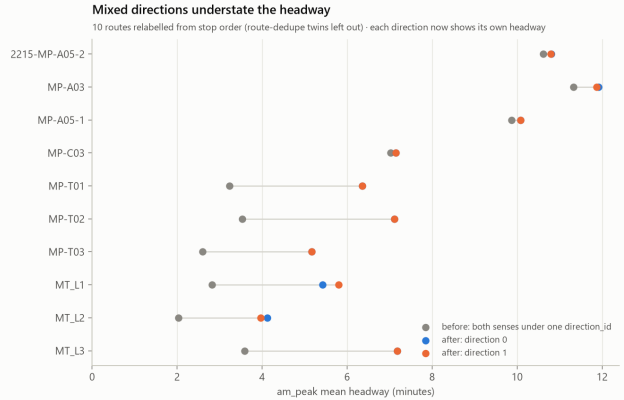


Read this carefully

The median is 8.0 min in every window. The feeds define uniform frequencies all day — a property of the source data, **not** a finding about the city.

STEP 8 Mixed directions: relabel from stop order

Some routes put both senses under one `direction_id` — headways then count both ways



Rule (only on mixed routes)

reference = the route's longest trip
sense(trip) = sign of the rank correlation of its stop order with the reference, over shared stops
label = the one most same-sense trips already carry; the other sense gets the other label

- 19 trips on 19 routes relabelled (10 improved, 9 mdb), listed in `direction_fixes.csv`
- `MT_L3`: published 420 s each way; mixed it read 230–330 s; now **420 s**
- Consistent routes keep the feed's labels; `--no-fix-directions` turns it off

STEP 9 Route–stop maps and route geometry

Three views of the same routes, built from different files

route shapes



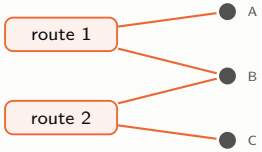
`shapes.txt` points in sequence —
the real drawn path, no stops, no time

route–stop links



consecutive stops from `stop_times` order,
`travel_min` = median across trips

route–stop map



which routes serve which stops —
the source of every "served?" test

```
for each trip, ordered by numeric stop_sequence, then stop_id:
    keep one stop per stop_sequence                  # mdb lists light-rail aliases under one number
    seg_s = next departure - this departure          # negative -> NULL (broken sequence)
group by (route, direction, stop, next_stop):
    travel_min = round(median(seg_s) / 60, 1)       n_obs = count of non-NULL seg_s
```

STEP 9 Can we trust each in-vehicle time?

Times are kept as published; `time_quality` says how much to trust them

Label	Rule	Arcs
`ok`	none of the below	24,013
`too_fast`	straight line between the stops $>$ 80 km/h (the real path is longer)	18
`too_slow`	route direction: shape km / median trip minutes $<$ 5 km/h	4
`zero`	0 s between two stops	1
`untimed`	no time	0

On the station network's 24,036 `in_vehicle` arcs; also on every timetable connection.

What the labels caught

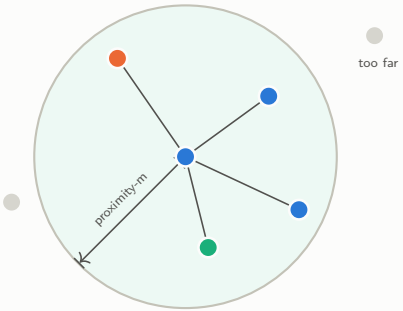
zig-zag: mdb `MT_L1`/`MT_L2` list each station twice (`MT-L1_xx`, `TL_0001_00xx`, 0 m) under one `stop_sequence`: 30 fake 0-minute arcs, now gone.

placeholders: every T16B branch is scheduled at 70 min whatever its length; the 3 km branches run at ≈3 km/h.

real: `T09-0` takes 110 min between two stops 150 m apart — a 26.6 km loop.

STEP 10 Find proximity pairs

Every pair of stops closer than `--proximity-m`, across all feeds



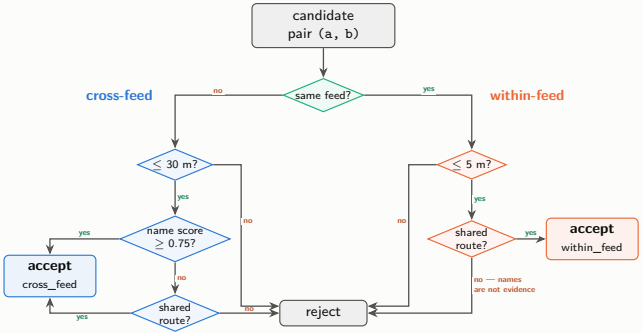
$$pairs = \{\, (a, b) : uid_a < uid_b,\ \ dist(a, b) \le r \,\}$$

Path	How
PostGIS	`ST_DWithin` on geography, GiST-indexed, on the `stops` table, filtered by feed
Local (`--dry-run`)	project to UTM, spatial-index join with `dwithin`

A trap dry runs cannot catch

The database path filters by feed id. Stops-only feeds must be in that list, or their rows match nothing — while the local path ignores the list and looks perfect.

STEP 11 Deduplicate stations: two rules, different evidence



Why names can't merge a within-feed pair

Two genuinely different stops of one feed can share a name — a long avenue, a plaza. Two stops of the **same route** within tens of metres are almost surely one station recorded twice. 5 m, not 50 m: the same route's stop across the street is 20–40 m away.

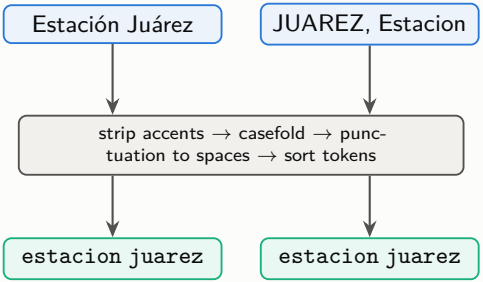
MiBici is never deduplicated

Stops-only sources are excluded from both rules: a dock is never an alias of a bus stop.

Accepted pairs form a graph: each connected component is one station.

STEP 11 The evidence functions

Name similarity



SequenceMatcher ratio = 1.0

Shared routes and the audit score

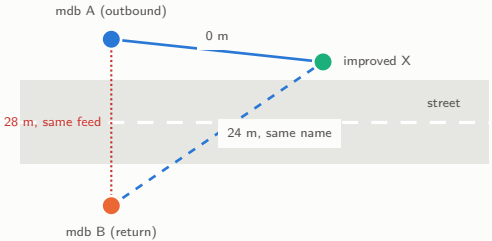
```
routes(stop)  = { normalize(route_short_name)
                  of routes serving the stop }
shared_routes = | routes(a) & routes(b) |

score = 0.5 * (1 - dist / radius_that_admitted)
      + 0.5 * name_score        # audit only
```

- Empty name ⇒ `name_score = 0`
- `score` is never a gate — it only helps review
- Every accepted pair records `match_kind`, so the two rules stay separately auditable

STEP 11 The same-feed conflict guard

Accepted pairs chain into groups — a group may not hold two far-apart stops of one feed

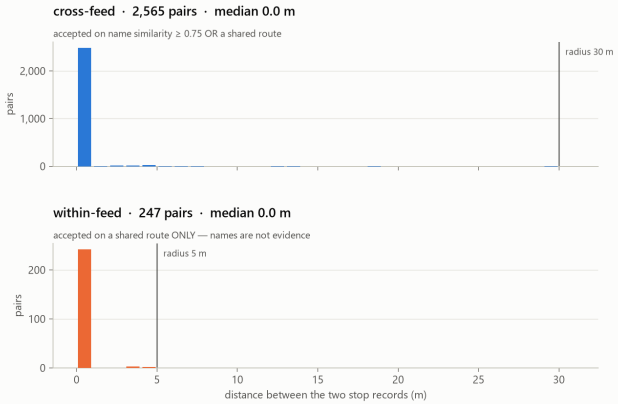


- Cross-feed pairs alone can chain A ∼ X ∼ B: two opposite platforms of one feed in one "station".
- Greedy union, closest pairs first: a pair is **rejected** if it would join two stops of the same feed more than 5 m apart.
- Rejected pairs go to `stop_equivalences_rejected.csv`; the stops stay neighbours (walk links).

772
pairs rejected

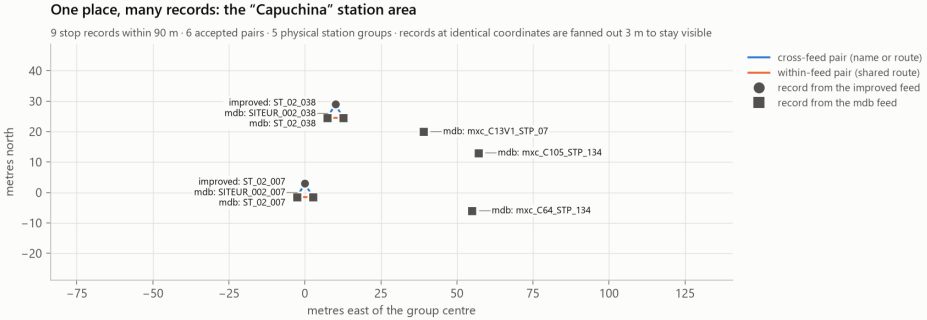
7.5 m
widest station

STEP 11 Dedupe measured: 2,812 accepted pairs



- **Cross-feed: 2,565 pairs, median 0.0 m.** Most are exact coordinate copies — the two feeds share a stop inventory.
- **Within-feed: 247 pairs, all $\leq$ 4.6 m.** One station recorded twice (`SITEUR_*` and `ST_*` light-rail ids).
- **1,604 groups**; 12,768 served records $\rightarrow$ **10,203 stations**. Median group span 0 m, widest 7.5 m.

STEP 11 One real place, many records



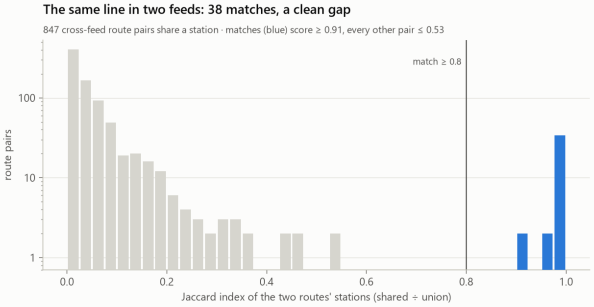
Both rules at once. Around one SiTren station, 9 records form 5 groups: `ST_02_007` matches across feeds (blue); `SITEUR_*` and `ST_*` match inside one feed on a shared route (orange). Nearby bus stops stay separate stations.

Opposite platforms stay apart

`ST_02_007` and `ST_02_038` are the two directions of one line, 26 m apart. With the 5 m rule and the conflict guard they are two stations joined by a walk link — no free U-turn when routing.

STEP 11b One source per line

A line published by two feeds would run twice in a timetable: double frequency, half the wait



Rule

stations(route) = its stops mapped to dedupe groups
match if Jaccard $\geq$ 0.8, best first, one-to-one
keep the feed with the later calendar

- Ids and short names can't decide: `2215-C98` vs `C98`; `L1` is both Mi Tren and SiTren
- **38 lines** in both feeds, all kept from mdb; improved keeps `MP-A05-2`, `ST_L1B`
- mdb lists every SiTren trip twice (`SITEUR_T_xx`, `ST_xx`): 8 dropped
- Routing inputs only; `routes.csv` keeps both with `duplicate_of`

STEP 12 Build route connectivity

Four ways two routes can meet — the strongest evidence names the edge



1. For each rule: `route_stops` ⋈ relation ⋈ `route_stops`
2. Drop self-pairs; order so `route_a` < `route_b`
3. Outer-merge the four sets on the route pair
4. Label with the **strongest** rule that produced it
5. `min_stop_distance_m` = 0 for `same_stop`, else closest proximity pair

Declared vs inferred

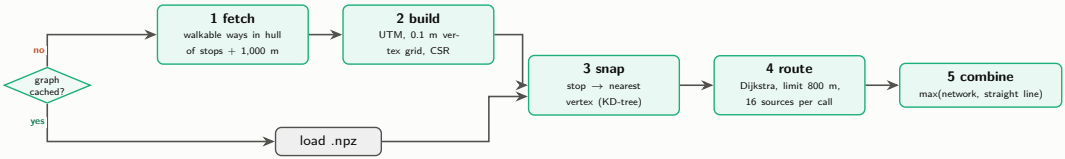
`same_station_dedup` stays separate from `same_parent_station`: one is inferred by the pipeline, the other declared by the operator.

249 routes, 7,902 pairs, 1 component.

`route_connection_stops.csv` lists *where*: every route pair and stop pair behind an edge.

STEP 13 Walk on real OSM streets

Straight-line distance understates walking — route it on the street network



Fetch & build

`highway` ways except motorway, motorway_link, proposed, construction. Endpoints snap to a 10 cm grid; ways connect only where they **share a coordinate** — bridges stay unconnected.

Snap & route

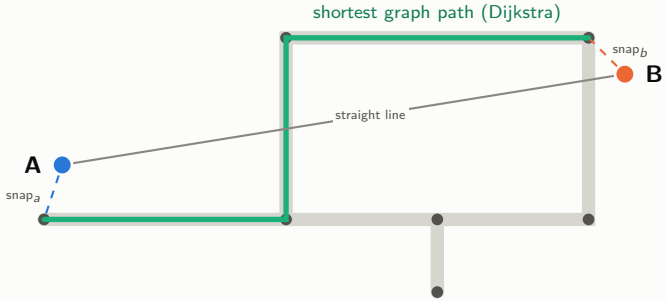
Nearest *vertex*, not nearest edge point. Snap > 200 m is labelled `unsnapped`. Each Dijkstra call allocates $16 \times 512{,}420$ floats ≈ 66 MB.

Cache key

Feed ids, hull buffer, OSM table and column, excluded highways, CRS — plus OSM row count, max `osm_id` and the stops' latest load time.

STEP 13 Inside the walk graph

How one pair of stops gets a walking distance



Combine

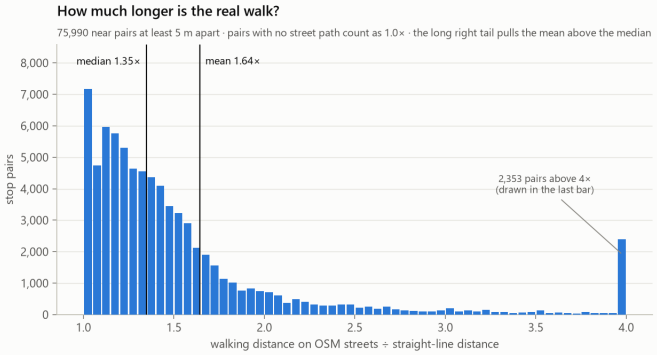
$network = path + snap_a + snap_b$

$distance = \max(network,\ straight\ line)$

no path within 800 m ⇒ keep the straight line, `walk_source = straight_line`

Applied **after** step 12 (connectivity stays geodesic) and **before** step 14 (walking minutes use real streets).

STEP 13 How much longer is the real walk?



1.35×

median walk / straight line

- 75,990 pairs at least 5 m apart; 74,689 of 77,706 routed on streets
- Mean 1.64×: a long tail of big detours
- 2,353 pairs above 4×

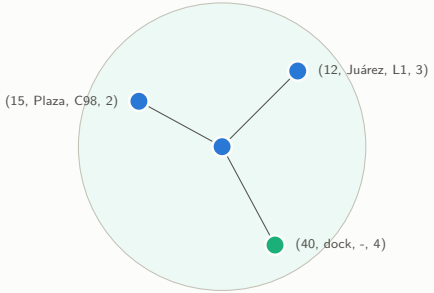
Use the median

The run log's "73 % detour" (1.73×) averages routed pairs only, including near-co-located stops whose tiny straight line inflates the ratio.

STEP 14 Near stations and same-line stations

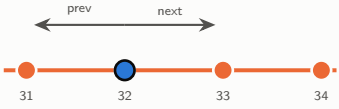
Two relations per stop, with deliberately different inclusion rules

near_stations — what can I walk to?



- Distance only, any line
- *minutes* = max(1, round(*metres*/83))
- Unserved neighbour listed with line –

same_line_stations — what's next?

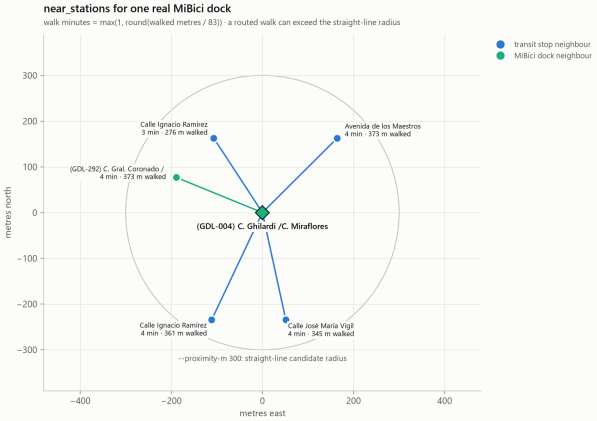


(31, Norte, L1, dir0, 1.5) (33, Centro, L1, dir0, 2.1)

- Sequence only, no distance limit
- Minutes = scheduled `travel_min`, else haversine at 25 km/h
- Previous *and* next stop, per line and direction

`ID` = 1..N in row order, referenced by every tuple; `stop_uid` is the cross-run key.

STEP 14 near_stations for one real MiBici dock



- Four transit stops and one dock, 3–4 minutes' walk
- Candidates are chosen by **straight line** (the circle)...
- ...but the reported distance is the **street walk** from step 13

So a walk can exceed the radius

Four neighbours are 345–373 m *walked* while sitting inside the 300 m straight-line circle.

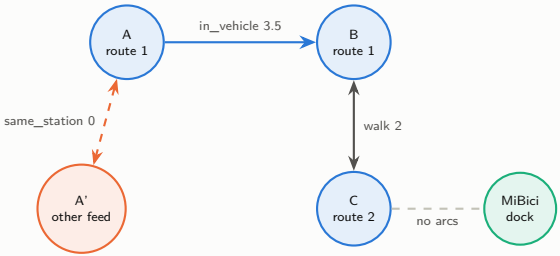
STEP 15 The quality sidecar: how each value was obtained

A measured 4 minutes and an estimated 4 minutes look identical — the sidecar tells them apart

Column	Values	Rule
`walk_source`	network, straight_line, mixed, unsnapped, none	label shared by the stop's neighbours; `mixed` if they disagree; `unsnapped` if snap > 200 m
`travel_source`	observed, estimated, mixed, none	scheduled `travel_min` vs the 25 km/h fallback
`travel_n_obs`	integer	minimum trips behind the stop's medians — the weakest evidence
`coord_confidence`	0.812, needs_review, no_match, unavailable	from `stop_osm_matches` (step 20)
`station_dedup`	canonical_uid or empty	the stop's dedupe group — **use it to count stations**

Same rows and IDs as `near_stations.csv`; the main file's format never changes because of it.

STEP 16 The station network: a routable graph of places




```
nx.shortest_path(G, a, b, weight="time_min")
```

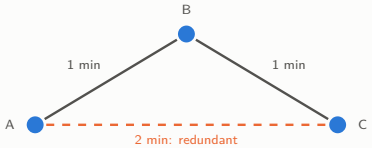
Edge	time_min	Built from
`in_vehicle`	`travel_min`	each route–stop link, one directed arc
`walk`	$\max(1, \mathrm{round}(m/83))$	each proximity pair, two arcs
`same_station`	0	each dedupe pair, two arcs

Riding, walking and transferring in one graph. MiBici docks are **unlinked nodes**: they were stepping stones that let walks chain past the 300 m limit.

`in_vehicle` arcs (route-dedupe winners)	24,036
`walk` arcs (72,286 pairs)	144,572
`same_station` arcs (2,812 pairs)	5,624
weak components among 12,284 transit stops	**1**

STEP 16 Many arcs never change a shortest time

Reported on every run; `--prune-station-network` drops them (off by default)



Walk pairs are removed **longest first**, only if the remaining walking network still links the two stops within the same minutes; alias pairs keep a spanning tree.

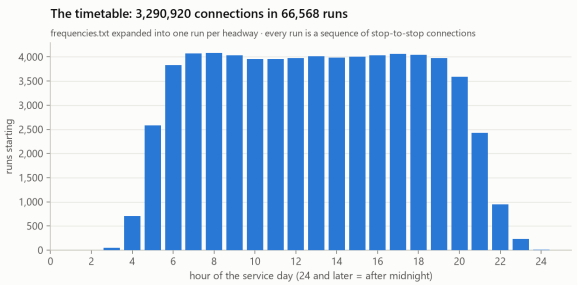
	kept	pruned
walk pairs	72,286	29,729
same_station pairs	2,812	2,565
all arcs	174,232	88,624

Verified

Dijkstra from 300 random stops: identical `time_min` distances for all 3.68 M reachable pairs. Walking metres along a path can grow.

STEP 16b The timetable

CSA, RAPTOR and time-dependent Dijkstra need every departure, not a median per stop pair



`connections.csv.gz` (gzip, 61 MB) — one row per vehicle hop, sorted by `dep_s`: `trip_id` (runs: `@HH:MM:SS`), `route_uid`, `direction_id`, `from_stop`, `to_stop`, `dep_s`, `arr_s`, `time_quality`

`footpaths.csv` — `self` change time (120 s) at 10,499 stops, 226 `same_station`, 101,018 `walk` (both ways); one hop, not transitively closed

Why it is large

66,568 runs × ≈49 stops = 3,290,920 connections: most mdb routes are “every N minutes” templates, expanded into real departures.

Every arc matches its connections; a CSA scan from 08:00 reaches 99.8–99.9 % of stops.

STEP 17 Export and persist

`transit_analysis.gpkg`

stops, near_links, route_shapes, route_stop_links, station_network

`route_connectivity.graphml`

+ route_connections.csv, route_connection_stops.csv (step 12)

`station_network.graphml`

+ nodes/edges CSVs: riding, walking, transfers (step 16)

`near_stations.csv`

walk and same-line relations, + quality sidecar (steps 14–15)

`stop_equivalences.csv`

dedupe pairs (+ rejected) (step 11)

`route_equivalences.csv`

lines in two feeds, duplicate trips, routes.csv duplicate_of (11b)

`connections.csv.gz`

+ footpaths.csv: the timetable (16b)

`route_*headways.csv`

headways (step 8), + direction_fixes.csv

`analysis_report, run_manifest`

dates, counts, quality; inputs, code hash, parameters (no secrets)

Close any QGIS layer from the GeoPackage before re-running: Windows locks the open file and the export fails.

The network on a toy model

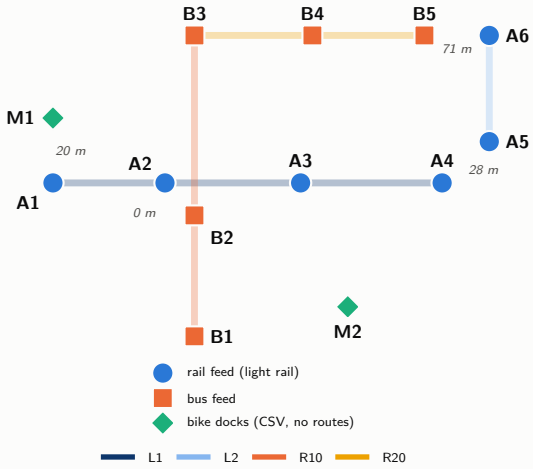
Three sources, five routes, 13 stops — what each step does to the graph

The model · which steps touch the graph · the assembled station network · summary

The toy model

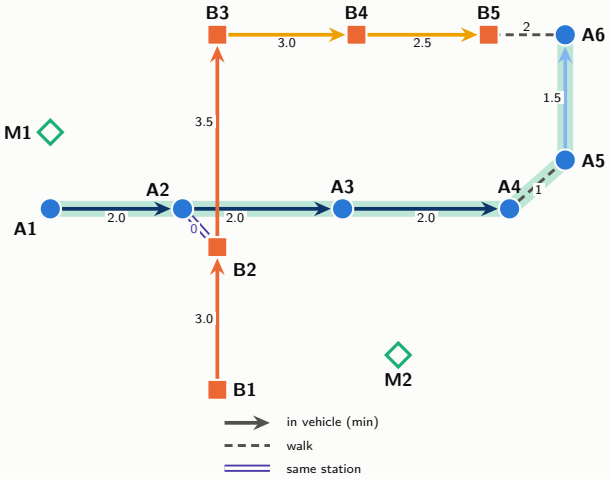
Three sources, five routes, 13 stops · analysed day: a Wednesday

`rail` — GTFS, light rail (`route_type 0`)	
Routes	L1: A1→A2→A3→A4 L2: A5→A6
Station	A4 and A5 share `parent_station` “Centro”
`bus` — GTFS, bus (`route_type 3`)	
Routes	R10: B1→B2→B3 (explicit trips)
	R20: B3→B4→B5 (`frequencies.txt`, every 10 min)
	R30: B3→B4 (Saturdays only)
Names	A2 = “Juárez”, B2 = “JUAREZ”, 0 m apart
`docks` — CSV `id, name, latitude, longitude`	
Stops	M1, M2 (no routes)



The toy station network

Everything assembled into one directed graph



Element	Count	From step
nodes (stops)	13	1–5
`in_vehicle` arcs	8	6, 9
`walk` arcs (2 pairs)	4	10, 11, 13
`same_station` (1 pair)	2	11
unlinked dock nodes	2	—

Shortest path A1 → A6

$$\text{A1} \xrightarrow{2+2+2} \text{A4} \xrightarrow{1} \text{A5} \xrightarrow{1.5} \text{A6} \;=\; \textbf{8.5 min}$$

via R10/R20: $2 + 0 + 3.5 + 3 + 2.5 + 2 = 13$ min

R30 runs on Saturdays only. A2 ≡ B2 by dedupe. The docks get no arcs: walking through a dock would chain two 300 m walks.

What each step does to the graph

Toy model: 13 nodes, 14 arcs in `station_network`; the timetable lists every run

Step	Nodes	in_vehicle	walk	same_station	timetable	Effect in the model
1–5 ingest	✓					13 stops, 2 docks unlinked later
6 service date		✓			✓	R30 disappears (Saturday only)
7 frequencies					✓	R20 becomes one run every 10 min
8 headways, directions						tables only (labels fixed if mixed)
9 route–stop links		✓			✓	8 timed arcs, one stop per sequence
10 proximity			✓		✓	candidate walk pairs
11 dedupe			✓	✓	✓	A2 ≡ B2: walk pair becomes same_station
11b one source per line		✓			✓	(a twin line in another feed would drop)
13 OSM walk			✓		✓	A6–B5: 1 → 2 min
16 station network	✓	✓	✓	✓		assembles the graph
16b timetable					✓	runs as connections + footpaths
18–22 Phase 3						interchanges, OSM checks — never change the graph

Phase 3 · Conflation

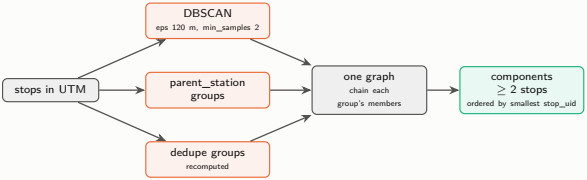
`detect-interchanges` and `conflate-osm`

18 Detect interchanges **19** Cross-check transfers.txt

20 Match stops to OSM transit nodes **21** Side of street **22** OSM coverage gaps

STEP 18 Detect interchanges

An interchange is infrastructure, not a schedule — whole feed, no service date



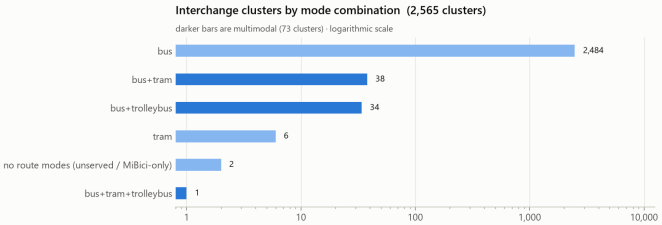

```
per cluster:
    modes                = distinct normalized route_type
    is_multimodal        = len(modes) > 1
    physical_stop_count = distinct canonical stations
    centroid, convex hull (EPSG:4326)
```

- A declared or aliased station is **never split**, even when its platforms are further apart than eps
- `route_type` normalized first: 700s → bus, 100s → rail, 400s → metro, 900s → tram
- `physical_stop_count = 1` flags a **phantom**: records of one station, not an interchange

Phase 2 vs Phase 3 dedupe

Phase 3 recomputes dedupe over the whole feed (no service date), so its route evidence and counts can differ from Phase 2's. Expected, not a bug.

STEP 18 Interchange clusters by mode



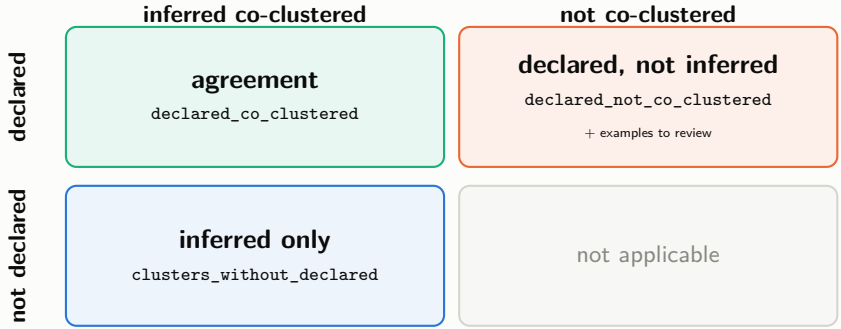
2,565
clusters

73
multimodal

150
single-station clusters

STEP 19 Cross-check transfers.txt

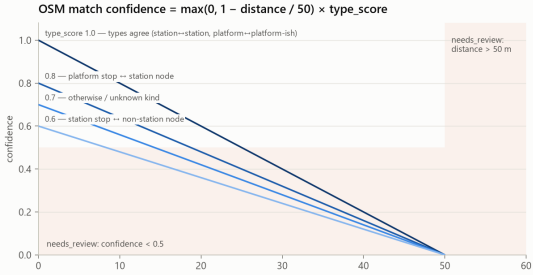
Do the clusters we inferred agree with the transfers the operator declared?



A feed without `transfers.txt` is listed in `feeds_without_transfers` — not an error.

STEP 20 Match stops to OSM transit nodes

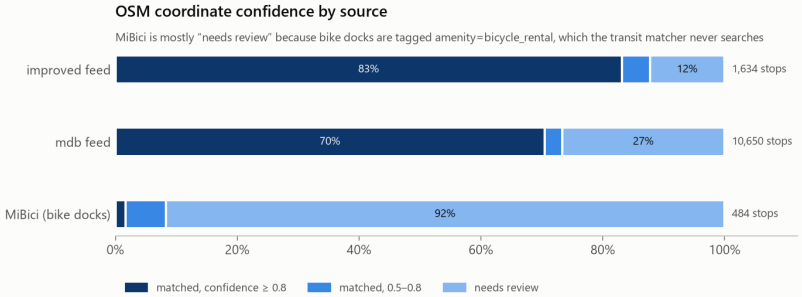
```sql
SELECT s.stop_uid, o.osm_id, o.kind, o.distance_m
FROM stops s
LEFT JOIN LATERAL (
  SELECT p.osm_id, ... AS kind,
    ST_Distance(s.geom::geography,
      ST_Transform(p.way,4326)::geography)
      AS distance_m
  FROM planet_osm_point p
  WHERE p.highway = 'bus_stop'
     OR p.public_transport IN
        ('platform','stop_position','station')
     OR p.railway IN ('station','halt','tram_stop')
  ORDER BY p.way <-> ST_Transform(s.geom, srid)
  LIMIT 1                          -- index KNN
) o ON TRUE;
```



- *confidence* $= \max(0,\ 1 - d/50) \times$ *type_score*
- `needs_review` $\Leftarrow$ no candidate, $d > 50$ m, or confidence < 0.5
- Schema validated first; fails fast listing every problem

STEP 20

OSM confidence by source



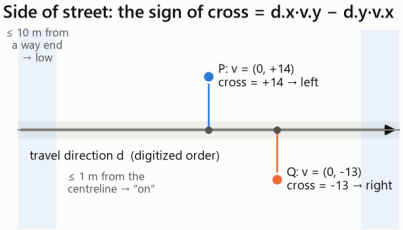
73.1%

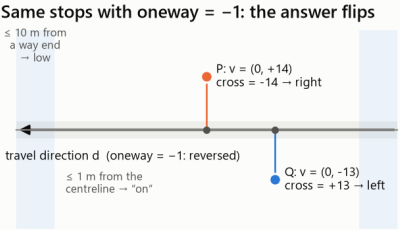
stops matched overall

MiBici's 92% "needs review" is a matcher-scope result

The query searches public-transport tags only. Bike docks are tagged `amenity=bicycle_rental` and are never searched, so a dock that sits exactly on its OSM node still comes back without a match. It is not evidence of bad coordinates.

STEP 21 Determine side of street





Rule

nearest highway centerline (KNN, ≤ 50 m) $\rightarrow$ nearest segment in UTM
d = direction *of travel* (reversed when `oneway = -1`)
v = stop minus its perpendicular foot
$cross = d_x v_y - d_y v_x$: > 0 left, < 0 right, $|v| \leq 1$ m on

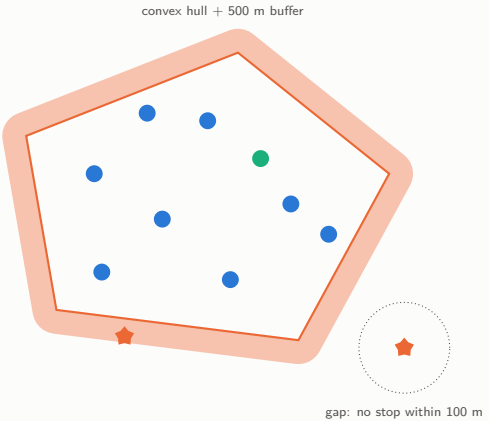
Low confidence when

side is `on` · foot within 10 m of a way end (an intersection) · $|v| < 2$ m

Two-way streets: side is relative to OSM digitization. Rail stations snap to the nearest road; centre-median BRT platforms land in `on`.

STEP 22 Find OSM coverage gaps

The reverse question: what transit does OSM know that the feeds don't?



Algorithm

1. *area* $= \mathrm{buffer}(\mathrm{hull}(\text{all stops}), 500\,\mathrm{m})$
2. Each OSM transit node inside the area
3. Nearest stop by KNN, measured geodesically
4. Keep it if that stop is $> 100\,\mathrm{m}$ away
5. Sort worst gap first

A **review list, not a verdict**: a gap is either a hole in the feeds or noise in OSM.

34

OSM-only transit nodes

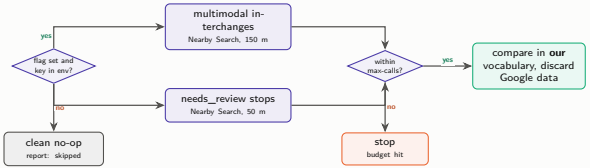
Phase 4 · External checks

Opt-in, budgeted, nothing stored but verdicts

24 Google QA and station locator

STEP 24 Google QA and station locator

Off by default — a flag *and* an API key in the environment are both required



Target	Verdicts
interchanges	CONFIRMED, PARTIAL, CONFLICT, NO_DATA
stops	STATION_FOUND, NOT_FOUND, NO_DATA

Licensing rails, enforced in code

Only the opaque `place_id` and the verdict are stored — a frozen column allowlist. No names, coordinates or lines, in the database, files or logs.

The station locator resolves station names and travel times, exports Excel, never writes to the database.

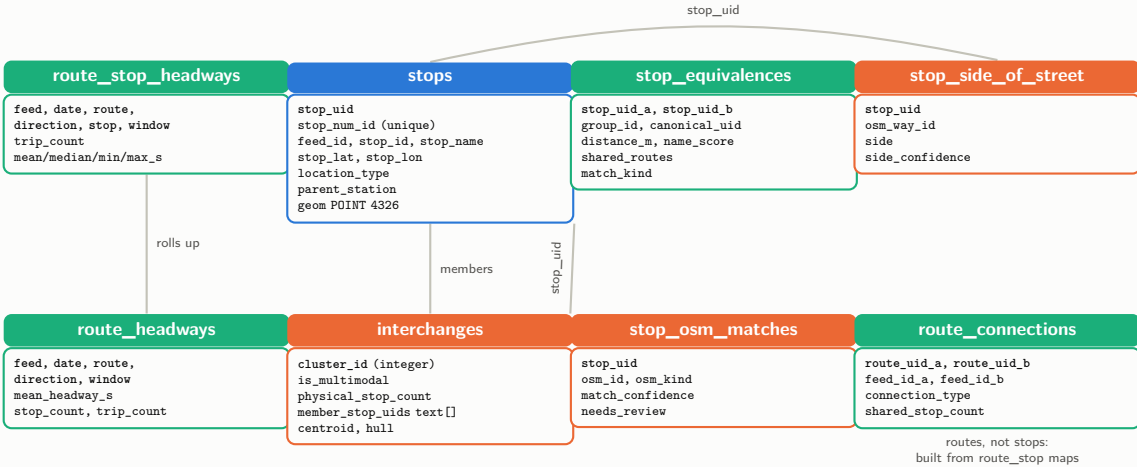
Results

The data model, the numbers, and how to read them

Data model · Guadalajara in numbers · choosing the proximity threshold
interpretation pitfalls · parameters · reproducing the run

The data model

Every analysis table keys back to `stops` — nothing is ever deleted from it



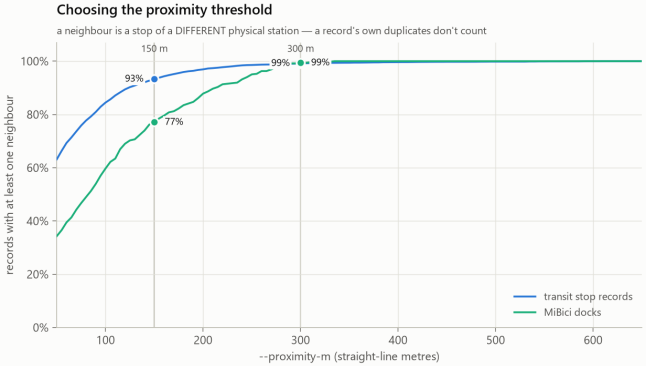
Guadalajara in numbers

Two GTFS feeds and the MiBici stops CSV · OSM network walk · 300 m proximity

14,349 stop records, 0 errors	**10,203** physical stations	**2,812** dedupe pairs (772 rejected)	**249 / 7,902** routing lines / route links
107,398 stop-level headway rows	**174,232** station-network arcs, 1 component	**1.35×** typical walk detour	**3.29 M** timetable connections
2,565 interchange clusters	**73.1%** stops matched to OSM	**67%** stops right of street (27% left)	**484** MiBici docks, 0 merged

Service dates: typical weekday 2021-05-04 (improved) and 2024-06-27 (mdb). The calendars never overlap, so cross-feed outputs combine two years; after route dedupe only 2 improved routes remain in routing.

Choosing the proximity threshold

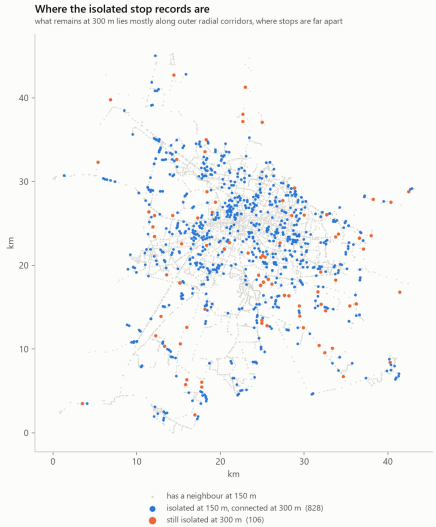


A neighbour is a stop of a **different** physical station — a record's own duplicates don't count.

	150 m	**300 m**
transit records with a neighbour	93%	99%
MiBici docks with a neighbour	77%	99%
isolated records	934	106

300 m ≈ a 3–4 minute walk.

Where the isolated stop records are



- **828** records were isolated at 150 m and connect at 300 m — mostly inside the urban area
- **106** remain isolated — mostly along outer radial corridors, where consecutive stops really are far apart

A threshold changes more than neighbour lists

`--proximity-m` also creates route `proximity` edges and station-network walk arcs. Compare connectivity figures only between runs with the same threshold.

Interpretation pitfalls

1. **Records are not stations.** Count with `COALESCE(station_dedup, stop_uid)`.
2. **A station is a platform group**: opposite platforms are separate stations, joined by walking.
3. **Check** `service_dates_coherent`: here the feeds' calendars never overlap.
4. `direction_id` **is unreliable** in the source; the pipeline relabels mixed routes.
5. **Count routes in routing** with `duplicate_of` empty: 249, not 287.
6. **Headways don't vary by time of day** because the source data doesn't.
7. **Use the median detour** ($1.35\times$), not the mean.
8. `needs_review` **for bike docks** is matcher scope, not bad coordinates.
9. **– in a near-stations tuple** marks a neighbour no line serves.
10. **Filter on** `time_quality` before trusting an in-vehicle time.
11. **Footpaths are one hop**: CSA needs them closed, or an algorithm that walks twice.

Parameters that shape the results

Parameter	Default	Step	Controls	Parameter	Default	Step	Controls
`--service-date` / `--date-rule`	typical weekday	6	day analysed	`--route-dedupe`	on	11b	one source per line
`--feed-service-date`	—	6	pin one feed	`--network-walk`	off	13	OSM streets
`--common-date`	off	6	one shared day	`--prune-station-network`	off	16	redundant arcs
`--window`	4	8	headway windows	`--min-transfer-s`	120	16b	change time
`--fix-directions`	on	8	relabel mixed routes	`--eps`	120	18	DBSCAN radius
`--transport-mode`, `--route`	all	6–9	routes kept	`--min-samples`	2	18	DBSCAN core
`--drop-unserved-stops`	on	4	never-served stops	`--match-threshold-m`	50	20	OSM match
`--proximity-m`	150	10–16	“near”	`--street-threshold-m`	50	21	centerline
`--dedupe-distance-m`	30	11	cross-feed radius	`--coverage-radius-m`	100	22	gap radius
`--dedupe-name-threshold`	0.75	11	name similarity	`--coverage-buffer-m`	500	22	service area
`--dedupe-within-feed-distance-m`	5	11	within-feed radius	`--max-calls`	200	24	Google budget

Fixed constants

walking 83 m/min · vehicle fallback 25 km/h · vertex grid 0.1 m · Dijkstra limit 800 m · snap label 200 m · at most 1,000 runs per frequency row · route match Jaccard 0.8 · `too_fast` 80 km/h, `too_slow` 5 km/h

Reproduce this run

```
FEEDS="improved-gtfs-mimacro-mitren-sitren.zip
       mdb-2366-202502241922.zip mibici.csv"

OUT=outputs/out_mibici_v12
python -m gtfs_ingest.cli ingest --feeds $FEEDS \
    --report $OUT --analysis-date 2026-10-31
python -m gtfs_ingest.cli analyze-frequency \
    --feeds $FEEDS --report $OUT \
    --network-walk --proximity-m 300
python -m gtfs_ingest.cli detect-interchanges \
    --feeds $FEEDS --report $OUT
python -m gtfs_ingest.cli conflate-osm --report $OUT
python -m gtfs_ingest.cli analyze-frequency ...   # pass 2

python docs/figures/make_figures.py --run $OUT
```

Go deeper (docs/guide/)

- `pipeline-guide.md`: every step and pitfall
- `workflow.md`: the data, by concern
- `data-dictionary.md`: every column
- `README.md`: install and operate

Two runs give byte-identical files; `run_manifest.json` records inputs, code hash and parameters. `--dry-run` skips the database.

What the outputs enable: routing

Two inputs, two families of algorithms

`station_network.graphml` — a static graph

Nodes = stops, arcs weighted by minutes (scheduled ride, walk, 0-min alias).

Dijkstra gives travel time *without waiting*: a lower bound, useful as a baseline. Frequencies could add an expected wait per boarding.

`connections.csv.gz` + `footpaths.csv` — a timetable

Every departure of the day, plus change and walk times.

CSA, **RAPTOR** and **time-dependent Dijkstra** give earliest arrival from a departure time, with waiting and transfers.

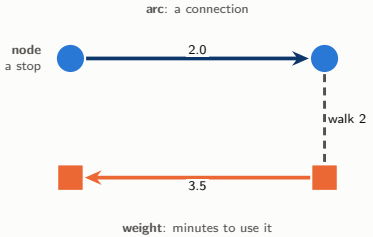
Already in place

one source per line · a typical weekday · directions fixed · time quality labels · deterministic outputs

Next

frequency-based graph (waiting cost) · access and egress from any point · a common algorithm interface · query sets and metrics · comparison with OpenTripPlanner / r5py

Glossary: graphs and connectivity



- A **graph** is a set of nodes joined by arcs; a ride runs one way, a walk both ways.
- A **shortest path** minimises the summed weights (Dijkstra).

Connectivity

Weakly connected component: nodes linked ignoring arc direction — the station network has 1.

Strongly connected: every node reaches every other following directions; route terminals reached one way only fall outside (9 stops here).

Bridge: an arc whose removal splits the graph.

DBSCAN (Phase 3)

Points with at least `min_samples` neighbours within `eps` metres are cores; clusters grow from cores. Isolated points are noise. Here: `eps` 120 m, `min_samples` 2.

Jaccard index

$|A \cap B| / |A \cup B|$: 1 = the same set, 0 = nothing shared (step 11b).